

**LAPORAN PRAKTIKUM APLIKASI
BERBASIS PLATFORM**

(PERTEMUAN 8)

**RUNNING MODUL, PENGENALAN FLUTTER,
PENGENALAN DART**



Disusun oleh:

NAJWA HUMAIRAH

2311102134

PSIF-11-REG04

Dosen Pengampu:

CAHYO PRIHANTORO, S.Kom., M.Eng

**PROGRAM STUDI S1 TEKNIK INFORMATIKA FAKULTAS
INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO**

2026

LINK GIT :

[Link Repository Pertemuan 8](#)

Modul 1

Running Modul

A. Git

Git merupakan perangkat lunak pengelola versi (Version Control System/VCS) yang banyak digunakan dalam proses pengembangan aplikasi dan perangkat lunak. Sistem ini dikembangkan oleh Linus Torvalds pada tahun 2005 dengan tujuan membantu pengembang dalam mengelola perubahan kode sumber secara efektif. Melalui Git, setiap modifikasi yang dilakukan pada file proyek dapat direkam secara sistematis sehingga riwayat perubahan dapat ditelusuri, dibandingkan, maupun dipulihkan ke kondisi sebelumnya apabila diperlukan.

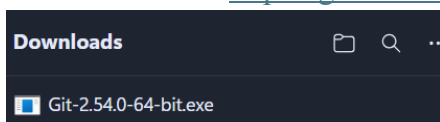
Dalam pengembangan perangkat lunak, pengelolaan versi menjadi aspek yang sangat penting, baik untuk proyek yang dikerjakan secara mandiri maupun secara kolaboratif. Git menyediakan mekanisme pencatatan aktivitas pengembangan yang memungkinkan pengguna mengetahui detail perubahan yang terjadi, termasuk identitas pengubah, waktu perubahan, serta tujuan dari modifikasi yang dilakukan. Dengan adanya fitur tersebut, proses pengembangan menjadi lebih terstruktur dan mudah dikendalikan.

Selain berfungsi sebagai alat pencatat perubahan, Git juga mendukung pengembangan beberapa versi proyek secara bersamaan. Pengembang dapat membuat cabang (branch) baru untuk menguji atau mengembangkan fitur tertentu tanpa memengaruhi versi utama yang sedang digunakan. Pendekatan ini membantu menjaga stabilitas aplikasi sekaligus mempercepat proses pengembangan.

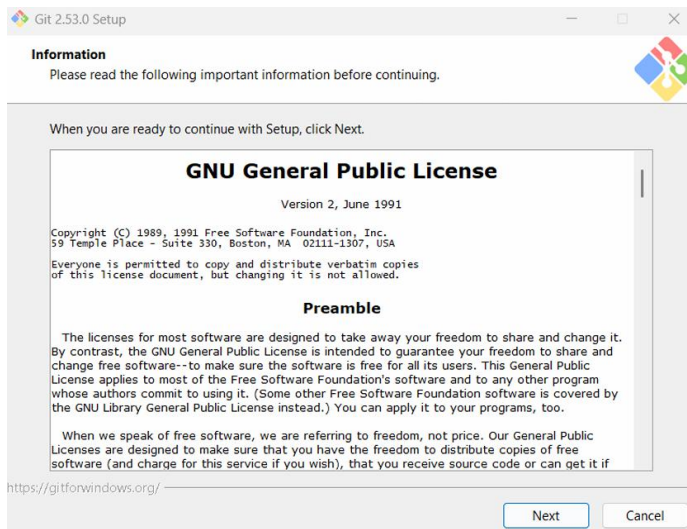
Salah satu karakteristik utama Git adalah penerapan konsep Distributed Version Control System (DVCS). Pada sistem ini, setiap anggota tim memiliki salinan lengkap repository beserta seluruh riwayat perubahannya di komputer lokal. Berbeda dengan sistem version control terpusat seperti Subversion (SVN) yang bergantung pada satu server utama, Git memungkinkan pengembang tetap bekerja meskipun tidak terhubung ke jaringan. Ketika koneksi tersedia, perubahan yang telah dilakukan dapat disinkronkan dengan repository jarak jauh (remote repository) melalui mekanisme push dan pull.

Berikut langkah-langkah instalasi Git:

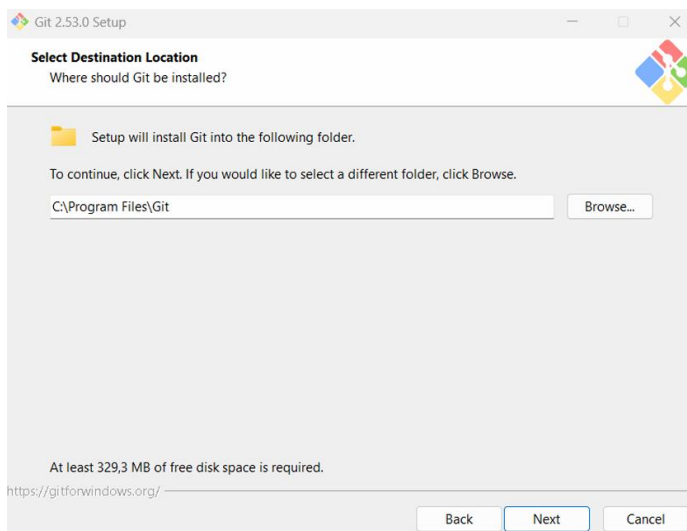
1. Download Git di <https://git-scm.com/install/windows>



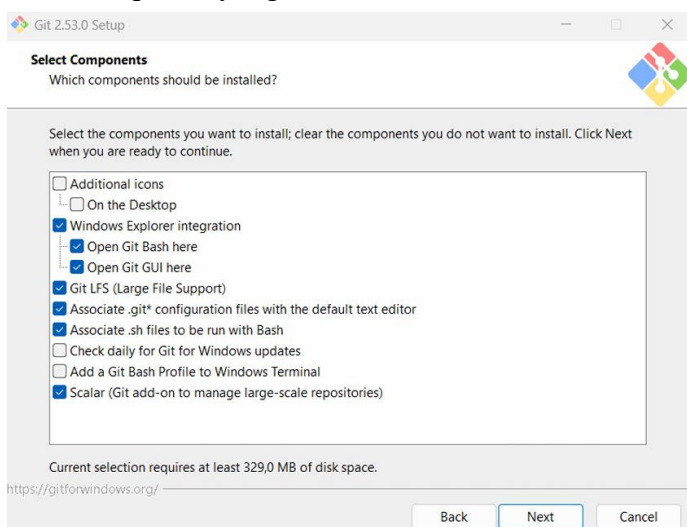
2. Klik 2 kali pada file .exe lalu next



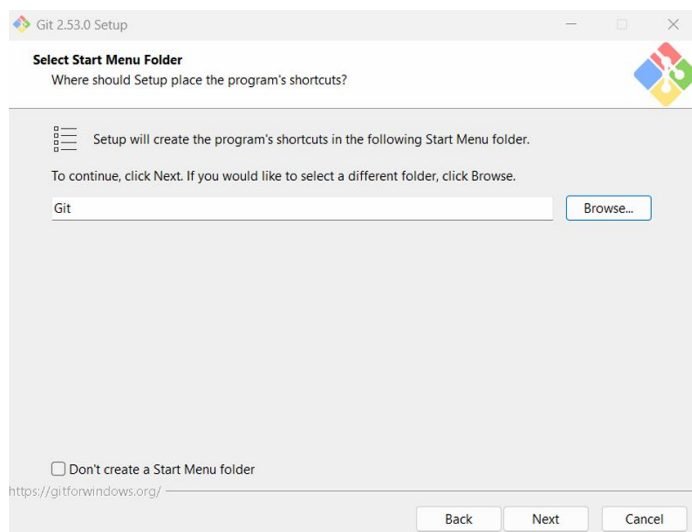
3. Pilih lokasi instalasi lalu next



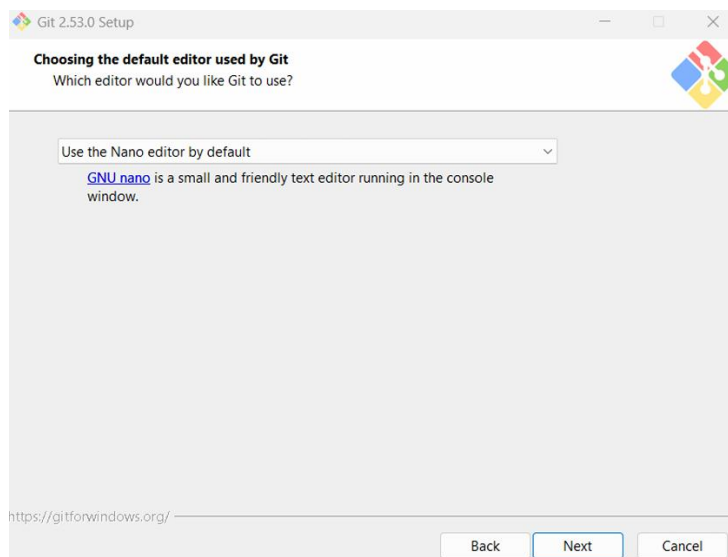
4. Pilih komponen yang akan diinstall lalu next



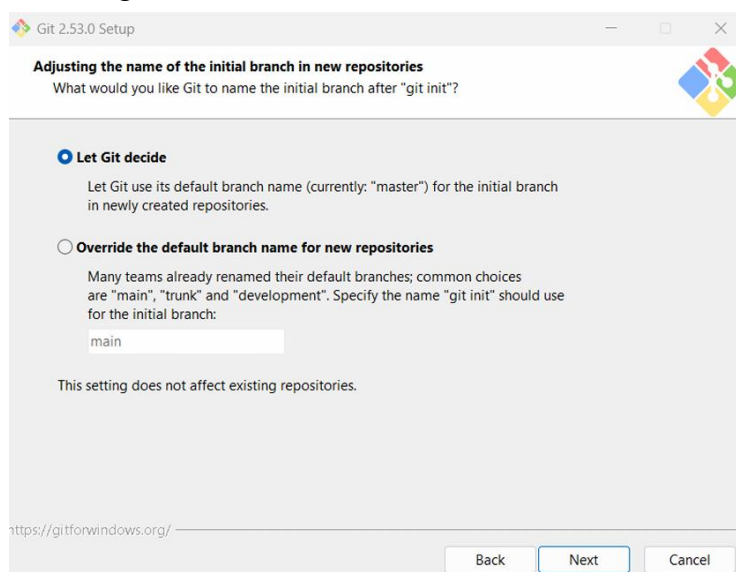
5. Select start menu folder lalu next



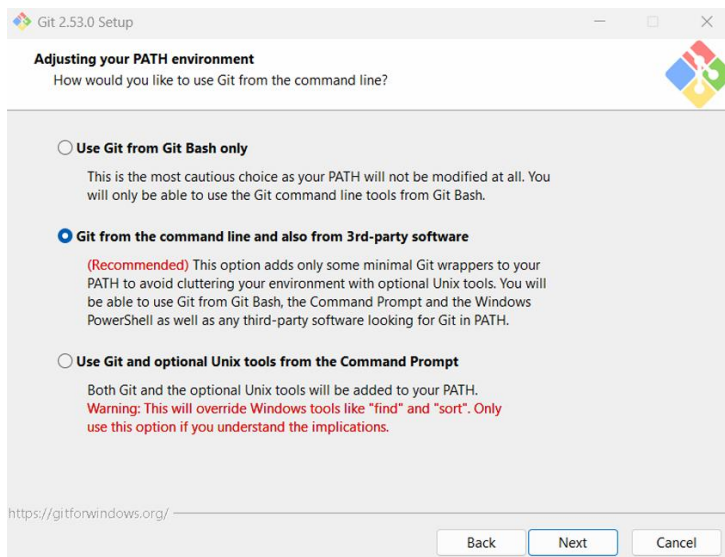
6. Pilih editor yang akan digunakan secara default oleh git lalu next



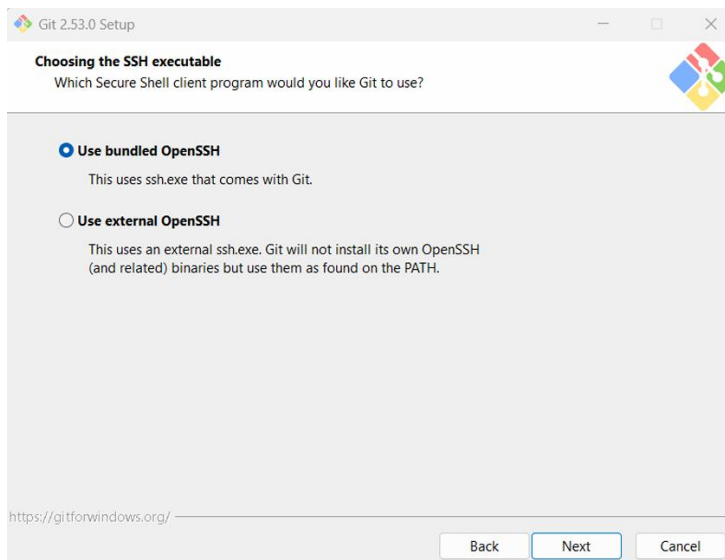
7. Pilih let git decide lalu next



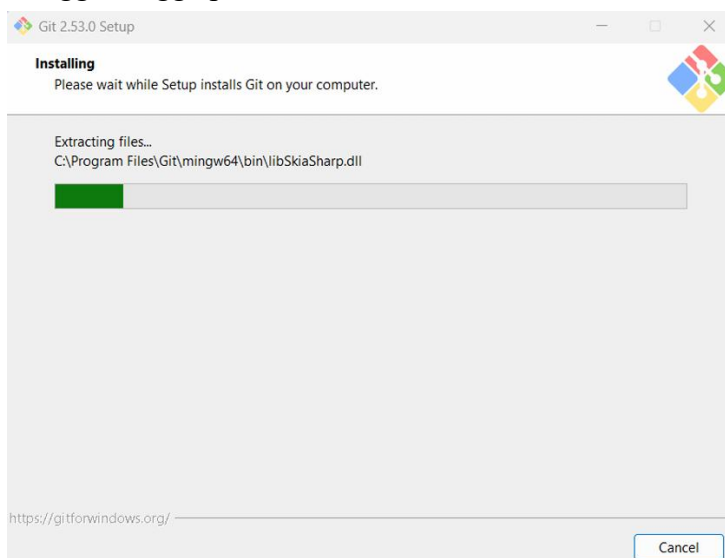
8. Pilih git from the command line and also from 3rd-party software, klik next



9. Untuk opsi selanjutnya lakukan secara default



10. Tunggu hingga proses instalasi selesai



11. Buka git bash kemudian cek versi git menggunakan command `git --version`, kemudian lakukan konfigurasi username (menggunakan command `git config --global user.name "username"`) dan email (menggunakan command `git config --global user.email email@contoh.com`). Kemudian cek konfigurasi yang dilakukan apakah sudah benar atau belum menggunakan command `git config--list`.

```
$ git --version
git version 2.43.0.windows.1
```

```
$ git config --global user.name "najwahmrh"
```

```
$ git config --global user.email "humairahnajwa3@gmail.com"
```

```
MINGW64:/c/Users/LENOVO iD
fluverryswa@DESKTOP-CHPQJGE MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
user.name=najwahmrh
user.email=humairahnajwa3@gmail.com
```

A. Instalasi JDK

Java Development Kit (JDK) merupakan paket perangkat lunak yang berfungsi sebagai lingkungan pengembangan untuk membuat aplikasi berbasis bahasa pemrograman Java. Di dalam JDK tersedia berbagai perangkat yang diperlukan selama proses pengembangan, termasuk compiler yang bertugas menerjemahkan kode sumber Java menjadi bytecode sehingga dapat dieksekusi oleh sistem Java Runtime Environment (JRE).

JDK memiliki beberapa komponen penting yang saling terintegrasi, antara lain:

- `javac` (Java Compiler), digunakan untuk mengonversi file sumber berekstensi `.java` menjadi file bytecode berekstensi `.class`.
- Java Runtime Environment (JRE), yaitu lingkungan yang menyediakan kebutuhan untuk menjalankan aplikasi Java.
- Java Virtual Machine (JVM), komponen yang bertanggung jawab mengeksekusi bytecode sehingga program dapat berjalan pada berbagai sistem operasi.
- Perangkat pendukung lainnya, seperti debugger, tools monitoring, dan utilitas pengembangan yang membantu proses pembuatan aplikasi.

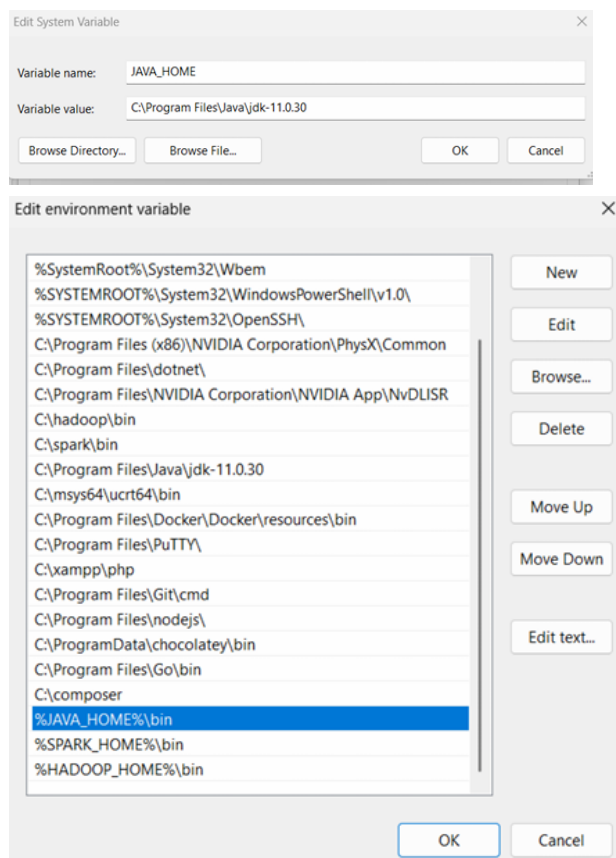
Mekanisme kerja JDK dimulai ketika programmer menulis kode program Java. Kode

tersebut kemudian dikompilasi oleh compiler menjadi bytecode yang bersifat universal. Selanjutnya, bytecode dijalankan oleh JVM yang terdapat dalam JRE. Proses ini memungkinkan Java menerapkan prinsip *platform independent*, sehingga satu program dapat dijalankan pada berbagai sistem operasi tanpa perlu dilakukan perubahan kode.

Sebagai teknologi yang dikembangkan oleh Oracle Corporation, JDK menjadi komponen fundamental dalam pengembangan perangkat lunak berbasis Java. Tanpa keberadaan JDK, proses kompilasi, pengujian, maupun pembangunan (*build*) aplikasi Java tidak dapat dilakukan. Oleh karena itu, instalasi JDK merupakan langkah awal yang wajib dipersiapkan sebelum memulai pengembangan aplikasi Java.

Berikut merupakan tahapan instalasi JDK yang perlu dilakukan:

1. Download JDK Installer melalui <https://www.oracle.com/java/technologies/downloads/>
2. Lakukan instalasi dengan default step (karena saya sudah pernah menginstall JDK sebelumnya pada mata kuliah Pemrograman Berorientasi Objek, maka saya tidak menginstall ulang lagi)
3. Tambahkan pengaturan path environment variable JDK



4. Cek versi jdk yang sudah diinstal pada CMD

```
fluverryswa@DESKTOP-CHPQJGE MINGW64 ~  
$ java --version  
java 23.0.2 2025-01-21  
Java(TM) SE Runtime Environment (build 23.0.2+7-58)  
Java HotSpot(TM) 64-Bit Server VM (build 23.0.2+7-58, mixed mode, sharing)
```

B. Instalasi Flutter SDK

Flutter SDK merupakan seperangkat perangkat lunak yang dirancang untuk mendukung proses pengembangan aplikasi lintas platform menggunakan framework Flutter. Dengan memanfaatkan SDK ini, pengembang dapat membuat aplikasi untuk perangkat mobile, web, maupun desktop hanya dengan satu basis kode yang sama. Flutter SDK dikembangkan oleh Google dan menggunakan bahasa pemrograman Dart sebagai fondasi utama dalam pembuatan aplikasi.

Di dalam Flutter SDK terdapat beberapa komponen penting yang bekerja secara terintegrasi. Salah satunya adalah Flutter Framework yang menyediakan berbagai widget dan pustaka untuk membangun antarmuka pengguna (User Interface/UI). Selain itu, terdapat Flutter Engine yang bertugas melakukan proses rendering tampilan aplikasi agar dapat ditampilkan secara optimal pada berbagai perangkat.

Flutter SDK juga dilengkapi dengan Dart SDK yang menyediakan compiler dan berbagai alat pendukung untuk menjalankan serta mengelola kode Dart. Tidak hanya itu, tersedia pula sejumlah command-line tools (CLI) seperti *flutter run*, *flutter build*, dan *flutter doctor* yang membantu pengembang dalam menjalankan, membangun, serta memeriksa konfigurasi lingkungan pengembangan Flutter.

Proses kerja Flutter dimulai ketika pengembang menulis kode program menggunakan bahasa Dart. Selanjutnya, Flutter SDK akan melakukan proses kompilasi menggunakan metode Ahead-of-Time (AOT) maupun Just-in-Time (JIT), sehingga menghasilkan aplikasi native yang dapat dijalankan pada berbagai platform dengan performa yang tinggi.

Salah satu keunggulan utama Flutter SDK adalah kemampuannya dalam mendukung pengembangan lintas platform. Dengan satu basis kode, aplikasi dapat dijalankan pada sistem operasi Android, iOS, web, maupun desktop tanpa perlu menulis kode yang berbeda untuk setiap platform. Pendekatan ini dapat menghemat waktu dan meningkatkan efisiensi proses pengembangan.

Selain itu, Flutter menyediakan fitur *hot reload* yang memungkinkan perubahan kode ditampilkan secara langsung tanpa perlu me-restart aplikasi. Fitur ini sangat membantu dalam mempercepat proses pengujian dan pengembangan karena pengembang dapat melihat hasil modifikasi secara real-time.

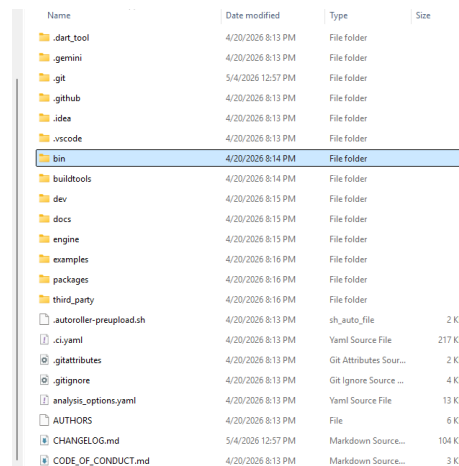
Flutter juga menawarkan fleksibilitas tinggi dalam pembuatan antarmuka pengguna melalui sistem widget yang dapat dikustomisasi sesuai kebutuhan. Dengan berbagai komponen UI yang tersedia, pengembang dapat menciptakan tampilan aplikasi yang modern, responsif, dan konsisten di berbagai perangkat.

Secara umum, Flutter SDK menjadi solusi yang efektif untuk membangun aplikasi

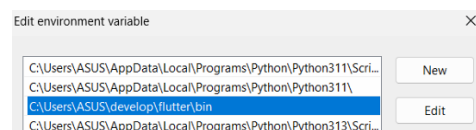
modern karena mampu menyederhanakan proses pengembangan, meningkatkan produktivitas, serta menjaga konsistensi pengalaman pengguna pada berbagai platform yang didukung.

Berikut langkah-langkah instalasi Flutter SDK:

- a. Download Flutter SDK secara manual pada link berikut <https://docs.flutter.dev/install/manual>
- b. Ekstrak berkas zip dan tempatkan folder flutter pada lokasi instalasi yang diinginkan untuk Flutter SDK



- c. Tambahkan pengaturan path environment variable flutter



- d. Jalankan Flutter Doctor.

```
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.44.2, on Microsoft Windows [Version
10.0.19045.6466], locale en-ID)
[✓] Windows Version (10 Home Single Language 64-bit, 22H2, 2009)
[!] Android toolchain - develop for Android devices (Android SDK version 36.1.0)
    ✗ cmdline-tools component is missing.
      Try installing or updating Android Studio.
      Alternatively, download the tools from
      Chrome - develop for the web/windows-android-setup for more details.
[✓] Visual Studio - develop Windows appst variable.
    ✗ Visual Studio not installed; this is necessary to develop Windows apps.
      Download at https://visualstudio.microsoft.com/downloads/.
      Please install the "Desktop development with C++" workload, including all
      of its default components
[✓] Connected device (3 available)
[✓] Network resources

! Doctor found issues in 2 categories.

f)uerrywa@DESKTOP-CHPJGE MINGW64 ~
```

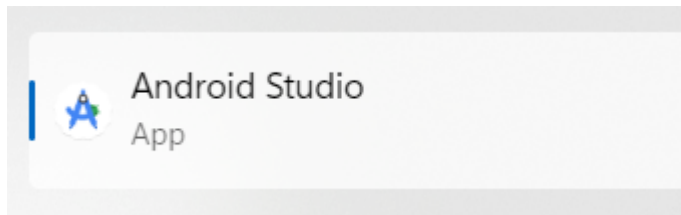
C. Instalasi Android Studio

Android Studio merupakan Integrated Development Environment (IDE) resmi dari Google yang digunakan untuk mengembangkan aplikasi Android. IDE ini menyediakan berbagai fitur terintegrasi yang membantu pengembang dalam menulis kode, merancang antarmuka, menjalankan, serta menguji aplikasi dalam satu lingkungan kerja.

Android Studio dilengkapi dengan berbagai komponen penting, seperti code

editor, Android SDK, emulator, dan sistem build berbasis Gradle. Komponen-komponen tersebut mendukung proses pengembangan aplikasi mulai dari penulisan kode hingga pengelolaan dependensi dan kompilasi aplikasi.

Dalam penggunaannya, pengembang menulis kode menggunakan Java atau Kotlin, kemudian aplikasi dibangun dan dijalankan melalui emulator maupun perangkat Android secara langsung. Dengan fitur yang lengkap dan terintegrasi, Android Studio mampu meningkatkan efisiensi serta produktivitas dalam pengembangan aplikasi Android.



D. Instalasi SDK Android

Android SDK (Software Development Kit) merupakan sekumpulan alat dan library yang digunakan untuk mengembangkan aplikasi Android. SDK ini menyediakan berbagai komponen penting, seperti API, build tools, emulator, dan debugging tools yang mendukung proses pembuatan serta pengujian aplikasi.

Android SDK dikembangkan oleh Google dan umumnya terintegrasi dengan Android Studio sebagai lingkungan pengembangan utama. Dengan SDK ini, pengembang dapat membangun aplikasi yang kompatibel dengan berbagai versi sistem operasi Android dan beragam jenis perangkat.

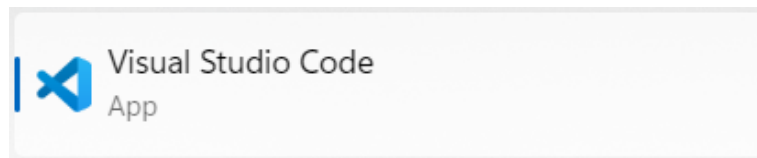
Android SDK dapat diinstal melalui Android Studio dengan membuka menu **Settings → Languages & Frameworks → Android SDK**.

E. Instalasi Visual Studio Code

Visual Studio Code (VS Code) adalah editor kode sumber yang dikembangkan oleh Microsoft untuk mendukung pengembangan berbagai jenis aplikasi. Editor ini mendukung banyak bahasa pemrograman dan dapat digunakan untuk pengembangan web, mobile, maupun backend.

VS Code menyediakan fitur seperti IntelliSense, debugging, terminal terintegrasi, serta dukungan ekstensi yang memungkinkan integrasi dengan berbagai tools dan framework, termasuk Flutter dan Git.

Berkat performanya yang ringan, fleksibel, dan mudah dikustomisasi, Visual Studio Code menjadi salah satu editor kode yang banyak digunakan oleh developer untuk meningkatkan efisiensi dalam proses pengembangan perangkat lunak.

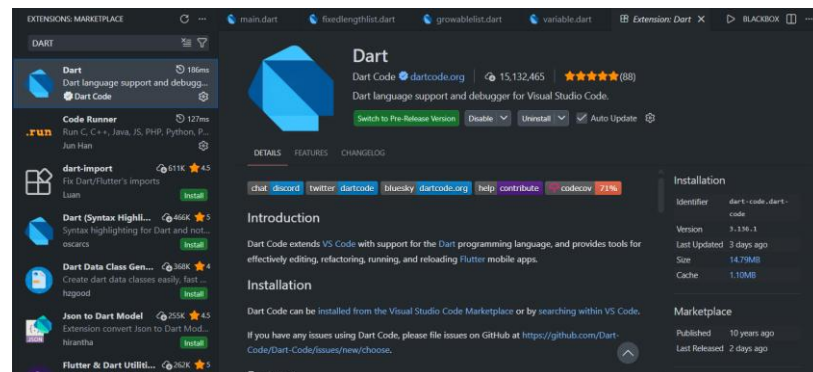


F. Instalasi Extension Visual Studio Code

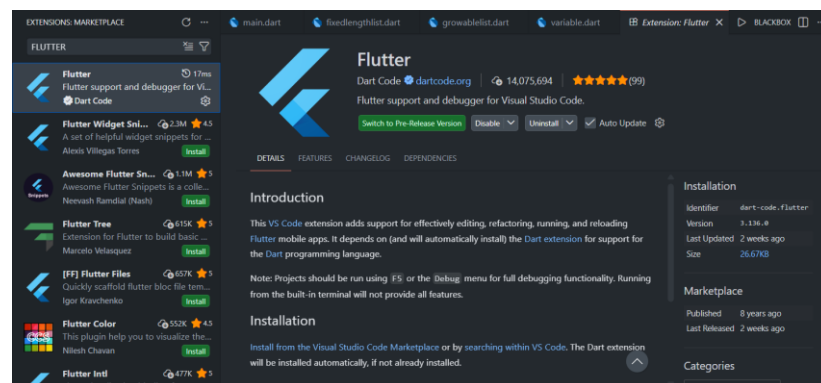
Beberapa extension sangat diperlukan untuk diinstall yang nantinya dipakai sebagai pendukung ketika membuat aplikasi menggunakan Flutter.

Berikut extension yang perlu diinstal:

1. Dart



2. Flutter



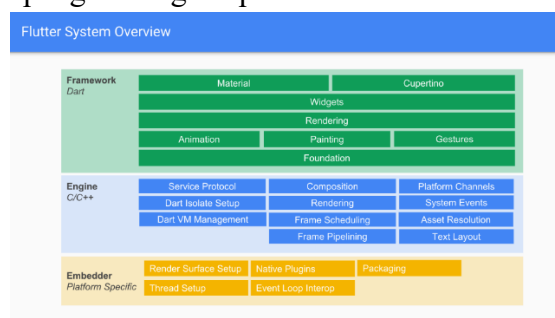
Modul 2

Pengenalan Flutter

Flutter merupakan framework UI yang dikembangkan oleh Google untuk membangun aplikasi lintas platform menggunakan satu basis kode. Framework ini menggunakan bahasa pemrograman Dart dan didukung oleh engine rendering yang mampu menghasilkan tampilan dengan performa tinggi.

Salah satu keunggulan Flutter adalah fitur *hot reload* yang memungkinkan perubahan kode langsung terlihat saat proses pengembangan tanpa perlu menjalankan ulang aplikasi. Selain itu, Flutter menggunakan kompilasi *ahead-of-time* (AOT) pada tahap produksi sehingga aplikasi dapat berjalan lebih cepat dan optimal.

Dengan performa yang baik serta dukungan untuk berbagai platform, Flutter menjadi salah satu framework populer dalam pengembangan aplikasi modern.



Dalam membangun antarmuka pengguna, Flutter menerapkan konsep *widget tree*, yaitu struktur hierarki yang menyusun tampilan aplikasi dari kumpulan widget yang saling terhubung. Setiap elemen pada UI direpresentasikan sebagai widget yang dapat ditempatkan di dalam widget lainnya.

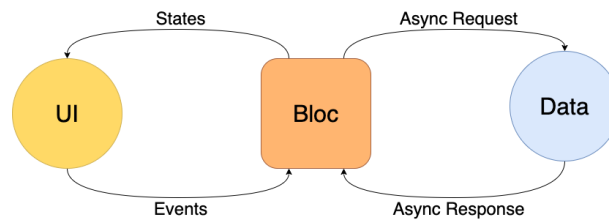
Flutter menyediakan dua jenis widget utama, yaitu *Stateless Widget* yang memiliki tampilan tetap dan *Stateful Widget* yang dapat berubah sesuai kondisi aplikasi. Konsep ini membantu pengembang dalam membuat antarmuka yang terstruktur, modular, serta lebih mudah dikelola.

A. Arsitektur Flutter

Selain Untuk mengelola alur data dan state aplikasi, Flutter mendukung berbagai pola arsitektur, salah satunya adalah BLoC (*Business Logic Component*). Arsitektur ini memisahkan logika bisnis dari tampilan (UI), sehingga kode menjadi lebih terstruktur dan mudah dikelola.

Pada BLoC, setiap *event* yang berasal dari pengguna atau sistem akan diproses untuk menghasilkan *state* baru. Selanjutnya, UI akan diperbarui berdasarkan perubahan state tersebut. Dengan alur *event* → *BLoC* → *state* → *UI*, pengelolaan data menjadi lebih terorganisir dan konsisten.

Penerapan BLoC memberikan beberapa keuntungan, seperti alur data yang jelas, kemudahan dalam pengembangan aplikasi berskala besar, serta logika bisnis yang lebih mudah diuji secara terpisah dari antarmuka pengguna. Oleh karena itu, BLoC sering digunakan untuk membangun aplikasi Flutter yang kompleks dan mudah dipelihara.



B. Hello World Pada Flutter

Pada Sebagai langkah awal dalam mempelajari Flutter, dibuat sebuah aplikasi sederhana *Hello World* untuk memahami struktur dasar proyek dan cara kerja framework Flutter. Program ini menampilkan teks "Hello World" pada layar sebagai contoh implementasi dasar antarmuka pengguna.

Berikut merupakan contoh kode program *Hello World* pada Flutter yang ditulis dalam file **main.dart**.

```
// ignore_for_file: prefer_const_constructors
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  // Root aplikasi
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Hello World",
      home: const MyHomePage(
        title: "Flutter Hello World Page",
      ),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({
    Key? key,
    required this.title,
  }) : super(key: key);
```

```

    }) : super(key: key);

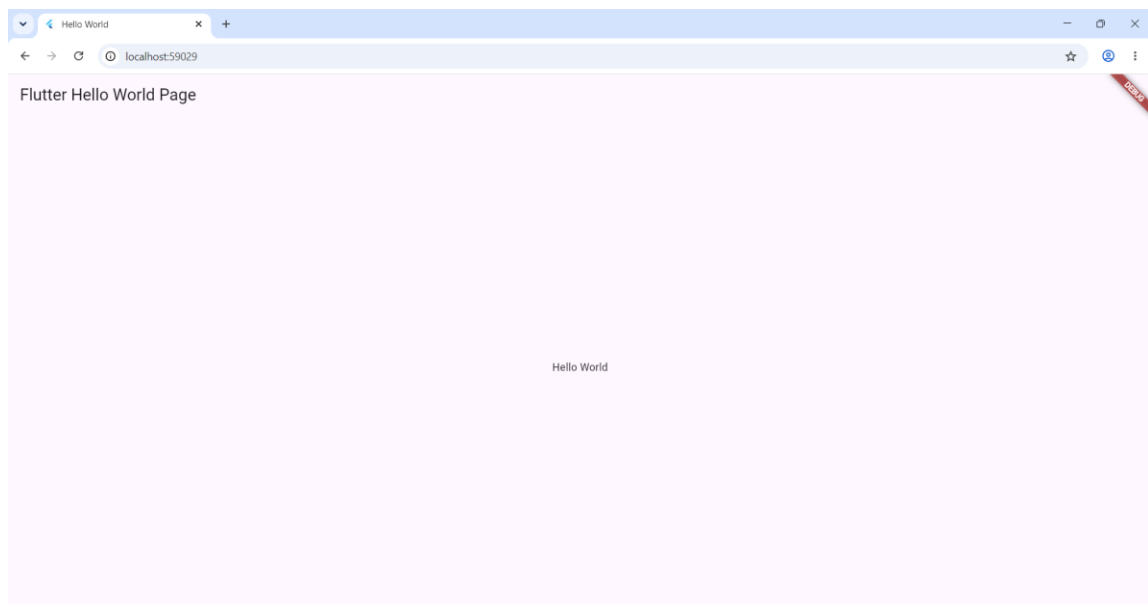
    final String title;

    @override
    State<MyHomePage> createState() => _MyHomePageState();
  }

  class _MyHomePageState extends State<MyHomePage> {
    @override
    Widget build(BuildContext context) {
      return Scaffold(
        appBar: AppBar(
          title: Text(widget.title),
        ),
        body: const Center(
          child: Text(
            'Hello World',
          ),
        ),
      );
    }
  }
}

```

Output :



Modul 3

Pengenalan Dart

Dart merupakan bahasa pemrograman yang digunakan sebagai dasar pengembangan aplikasi Flutter dan dikembangkan oleh Google. Meskipun tidak perlu menguasainya secara mendalam di awal, pemahaman mengenai konsep dasar seperti variabel, percabangan, perulangan, list, dan fungsi sangat membantu dalam proses pengembangan aplikasi.

Secara sintaks, Dart memiliki kemiripan dengan bahasa pemrograman seperti C dan Java, termasuk penggunaan kurung kurawal ({}) untuk blok kode serta tanda titik koma (;) di akhir pernyataan. Selain itu, Dart juga mendukung konsep *Object-Oriented Programming* (OOP) yang memungkinkan pembuatan kode yang lebih terstruktur dan mudah dikelola.

Dengan memahami dasar-dasar Dart, pengembang dapat mulai membangun aplikasi Flutter dengan lebih efektif dan meningkatkan pemahaman bahasa tersebut seiring berjalannya proses pengembangan.

- Variabel

Untuk penggunaan variable di dart, terdapat beberapa cara, yaitu dengan var, type annotation dan multiple variable.

```
//var
var variableName;
var name = "Deka";

//type annotation
String? nama;
String namaLengkap = "Najwa Humairah";
int? a, b, c;

//multiple variable
int x = 1, y = 2, z = 3;
```

Variable primitive yang tersedia di dart : integer, double, string, boolean

- Statement Control

Terdapat beberapa cara untuk mendeklarasikan statement control, yaitu if, if else, if else if, switch case.

- IF Statement

```
void main() {
  int angka = 6;

  //IF Statement
```



```
if (angka % 2 == 0) { //condition
    print("Angka Genap"); //statements
}
}
```

- IF ELSE Statement

```
void main() {
    int angka = 7;

    //IF Statement
    if (angka % 2 == 0) { //condition
        print("Angka Genap"); //statements
    } else {
        print("Angka Ganjil"); //statements
    }
}
```

- IF ELSE IF Statement

```
void main() {
    int nilai = 73;
    if (nilai >= 85) {
        //condition 1
        print("Grade A"); //statements 1
    } else if (nilai >= 75) {
        //condition 2
        print("Grade B"); //statements 2
    } else if (nilai >= 65) {
        //condition 3
        print("Grade C"); //statements 3
    } else {
        //else
        print("Grade D"); //statements 4
    }
}
```

- Switch Case

```
void main() {
    String hewan = "Ayam";
```

```
switch (hewan) {  
    //switch(expression)  
    case "Kucing": //case value 1  
        print("berkaki 4"); //statements  
        break;  
    case "Ayam": //case value 2  
        print("berkaki 2"); //statements  
        break;  
    default:  
        print("tidak valid"); //statements  
        break;  
}  
}
```

- **Looping**

Secara umum, terdapat dua cara untuk melakukan looping di dart, yaitu menggunakan for loop dan while loop.

- **FOR loops**

Gunakan for loop saat kondisinya tau persis seberapa banyak looping akan dilakukan, contohnya melakukan perulangan sebanyak 10 kali dengan iterasi sebanyak 1 tingkat atau 1 kali.

```
void main() {  
    for (int angka = 0; angka < 10; angka++) {  
        print("Angka = " + angka.toString());  
    }  
}
```

- **WHILE Loops**

Gunakan while loop saat kondisinya tidak tahu kapan perulangan akan berhenti, contohnya sediakan input angka hingga user menginput tanda "-".

```
void main() {  
    int angka = 0;  
    while (angka < 5) { //expression  
        print("Angka = " + angka.toString()); // Statement(s) to be executed if  
        // expression is true  
        angka++;  
    }  
}
```

- List

Secara umum, kumpulan banyak data dalam satu variable disebut array. Tetapi beberapa bahasa pemrograman menyebutnya dengan list, termasuk bahasa dart ini. List memiliki 2 tipe, yaitu Fixed Length List dan Growable List.

- Fixed Length List

Tipe list ini memiliki panjang index yang tetap dan tidak dapat bertambah banyak.

```
void main() {  
  var listAngka = List.filled(3, 0);  
  listAngka[0] = 15;  
  listAngka[1] = 44;  
  listAngka[2] = 98;  
  print(listAngka);  
  var listNama = List.filled(3, "");  
  listNama[0] = "Najwa";  
  listNama[1] = "Humairah";  
  listNama[2] = "Alya";  
  
  print(listNama);  
}
```

- Growable List

Gunakan growable list apabila memiliki banyak object yang tidak menentu atau banyaknya object yang terus bertambah.

```
void main() {  
  List<int> listAngka = [];  
  
  listAngka.add(10);  
  listAngka.add(20);  
  listAngka.add(30);  
  listAngka.add(40);  
  
  print(listAngka);  
  
  List<String> listNama = [];  
  
  listNama.add("Najwa");  
  listNama.add("Humairah");  
}
```

```
listNama.add("Alya");

print(listNama);
}
```

- Fungsi

Dalam Dart, fungsi digunakan untuk membagi program menjadi bagian-bagian kecil yang memiliki tugas tertentu sehingga kode lebih terstruktur dan mudah dikelola. Penggunaan fungsi juga membantu mengurangi pengulangan kode serta meningkatkan keterbacaan program.

Untuk menghasilkan kode yang berkualitas, pengembang umumnya menerapkan prinsip-prinsip seperti SOLID, KISS (Keep It Simple, Stupid), dan YAGNI (You Aren't Gonna Need It). Prinsip-prinsip tersebut menekankan pemisahan tanggung jawab (separation of concern), sehingga setiap bagian kode memiliki peran yang jelas dan spesifik.

Dengan memanfaatkan fungsi dan menerapkan prinsip pengembangan yang baik, aplikasi menjadi lebih mudah diuji, dipelihara, serta dikembangkan seiring bertambahnya kompleksitas proyek.

- Mendefinisikan Fungsi

```
String cekGanjilGenap(int n) {
}
```

- Memanggil Fungsi

```
int angka = 8;
String hasil = cekGanjilGenap(angka);
print("Angka " + angka.toString() + " adalah " + hasil);
```

- Mengembalikan Nilai

Tambahkan return apabila mendefinisikan sebuah fungsi, contohnya ada pada codingan dibawah yang bisa mengembalikan nilai "Ganjil/Genap" dari angka yang sudah ditentukan.

```
String cekGanjilGenap(int n) {
  if (n % 2 == 0) {
    return "Genap";
  } else {
    return "Ganjil";
  }
}
```

- Menambahkan Parameter

Fungsi memiliki scope yang terbatas, tentunya fungsi butuh input dari luar agar program didalamnya bisa memproses tugasnya.

```
String cekGanjilGenap(int n) {  
    if (n % 2 == 0) {  
        return "Genap";  
    } else {  
        return "Ganjil";  
    }  
}  
  
void main() {  
    int angka = 8;  
    String hasil = cekGanjilGenap(angka);  
    print("Angka " + angka.toString() + " adalah " + hasil);  
}
```

Pada fungsi diatas, n merupakan parameter. Variable diluar fungsi yang dibuat agar dapat digunakan didalam fungsi